Virtualization and Containerization

21 December 2025 13:56

Virtual Machine: (VM)

The goal of devops engineer is to use the resources efficiently.

A Virtual Machine server is one physical computer that behaves like many separate computers at the same time.
Each of those “separate computers” is called a Virtual Machine (VM).

Real-life example

Think of a large apartment building 🏢
The building = one physical server (real hardware)
Each flat = one Virtual Machine
Each family lives independently in their flat

Same way:

In computer terms

A VM:

Has its own Operating System (Windows, Linux)
Has its own apps
Has its own memory and CPU share
Works independently

Virtualization:
Virtualization is the technology that allows one physical computer to act like many computers.

Why is a Virtual Machine Server important?

1. Saves money

Instead of buying:

- 10 physical computers ❌

You buy:

- 1 powerful server
- Run 10 VMs on it ☑

2. Better use of resources

Without VMs:

- One server may use only 20% power

With VMs:

- CPU, RAM, and storage are fully used
- No waste

3. Easy backup and recovery

- A VM is just files
- You can copy, backup, restore quickly

- If one VM crashes, others are safe

4. Safe isolation

If one VM gets a virus:

- Other VMs are not affected

Just like fire in one flat does not destroy the whole building.

5. Easy testing

Developers can:

- Create a VM
- Test software
- Delete it after use

No risk to the main system.

3. What is a Hypervisor?

A **Hypervisor** is the **boss software** that creates, controls, and manages Virtual Machines.

In other words,
A **Hypervisor** is the **manager** that creates and controls Virtual Machines.

Real-life example: (School Principle example)

Apartment example again:

- Building owner / caretaker = **Hypervisor**
- Decides:
  - Which flat gets more electricity
  - Who can enter
  - Which flat is powered on or off

Without the caretaker, flats cannot exist.

What Hypervisor does

- Creates VMs
- Starts and stops VMs
- Allocates CPU and RAM
- Keeps VMs isolated

5. Types of Hypervisors

Type 1 (Bare-metal)

Runs directly on hardware.
Example:

- VMware ESXi
- Microsoft Hyper-V
- KVM

Real-life:
Caretaker lives in the building itself. Very efficient.
Used in data centers.

Type 2 (Hosted)

Runs on top of an OS like an app.
Example:

- VirtualBox
- VMware Workstation

Real-life:

Caretaker lives outside and comes when needed.
Used on laptops and desktops.

6. Virtual Machine vs Physical Machine

Physical Machine	Virtual Machine
One OS only	Multiple OS
Expensive	Cost-effective
Hard to scale	Easy to create
Hardware bound	File-based

Key difference (very important)

Virtual Machine	Container
Has full OS	Shares host OS
Heavy	Lightweight
Slow startup	Starts in seconds
Uses more RAM	Uses less RAM

Visual in words

Physical Server
↓
Hypervisor
↓
VM 1 (Linux + App)
VM 2 (Windows + App)
VM 3 (Linux + App)

Why companies love Virtual Machines

- Saves money on hardware
- Easy to create new servers in minutes
- Better security (VMs are isolated)
- Easy backup and restore
- Better disaster recovery

Example:
If one VM crashes → others keep running

5. How everything works together (big picture)

1. Physical server (hardware)
2. Hypervisor installed on it
3. Hypervisor creates multiple Virtual Machines
4. Each VM runs its own OS and apps

1. Problems with Virtual Machines (VMs):

Virtual Machines are powerful, but they come with real drawbacks.

Problem 1: Heavy and slow

Each VM has:

- Its own Operating System
- Its own memory and disk
- Its own startup process

That means:

- Takes minutes to start
- Consumes a lot of RAM and storage

Real life analogy:

It is like **buying a full house for every guest**, even if they stay only one night.

Problem 2: Wasted resources

If your app needs:

- 200 MB RAM
  But VM has:
- 2 GB RAM assigned

The rest is mostly wasted.

Problem 3: Hard to scale quickly

If traffic suddenly increases:

- Creating a new VM takes time
- OS boot, patches, setup

Not ideal for modern apps where users come and go quickly.

Problem 4: "Works on my machine" problem

App works on:

- Developer laptop ✗
- Fails in testing ✗
- Breaks in production ✗

Because:

- Different OS versions
- Different libraries
- Different configurations

2. Why Virtualization alone is not enough today

Virtualization was great when:

- Apps were big
- Changes were slow
- Scaling was predictable

Modern apps are:

- Microservices based
- Frequently updated
- Need instant scaling

That's where VMs struggle.

Container Technology

What is Containerization?

Containerization is a way to run applications by packaging:

- App code
- Libraries
- Dependencies

Together, **without a full OS**.
The most popular container platform is **Docker**.

Containers can be created in two ways.

1. On Top of VM
2. On Top of Physical Server.

Problem → Solution mapping

VM problem: Slow startup

Container solution:
Containers start in **seconds**, sometimes milliseconds.
Example:

- VM start: 2–5 minutes
- Container start: 1–2 seconds

VM problem: High resource usage

Container solution:
Containers share the same OS kernel.
Meaning:

- 10 containers ≈ cost of 1 OS
- 10 VMs = 10 OS copies

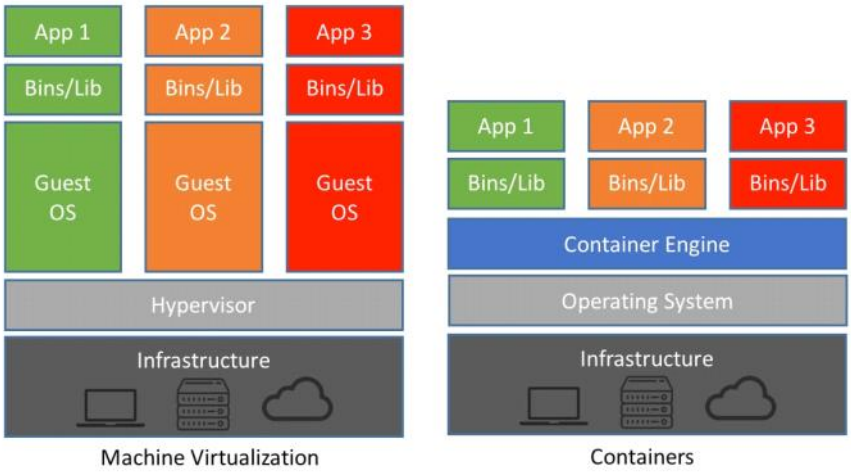
VM problem: Environment mismatch

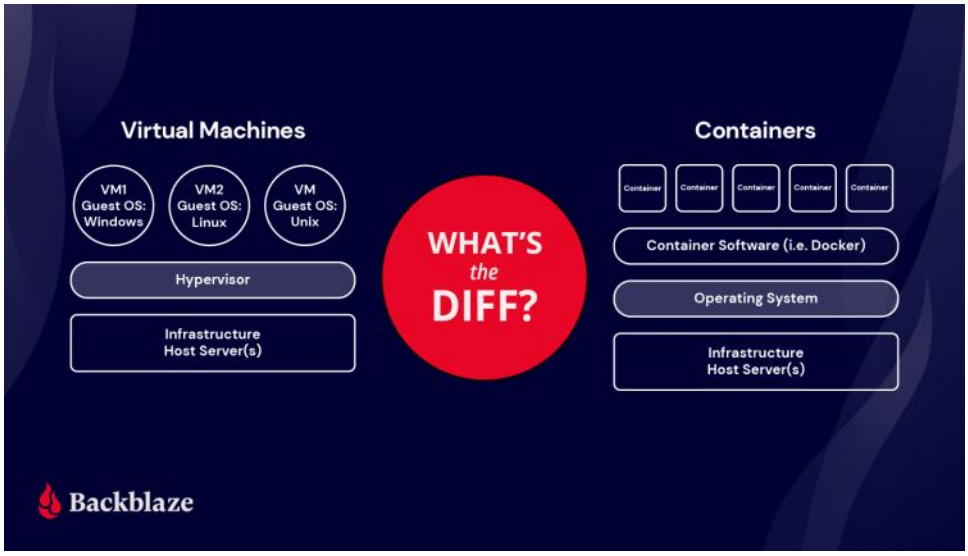
Container solution:
"Build once, run anywhere"
If it runs in Docker:

- Laptop ✓
- Test server ✓
- Production ✓

VM problem: Scaling difficulty

Container solution:
Spin up containers instantly based on traffic.





Real-time example (best way to understand)

Scenario: Online Food Delivery App 🍔

Your app has:

- Login service
- Order service
- Payment service

Using only Virtual Machines

- Each service runs in its own VM
- Traffic spike at lunch time
- Need more order VMs
- VM startup takes minutes
- Users face delays

Result:

- Slow scaling
- Poor user experience

Using Containers (Docker)

- Each service runs in a container
- Lunch time traffic spikes
- Platform instantly creates 10 more containers
- Containers start in seconds
- After lunch, containers shut down

Result:

- Fast
- Efficient
- Cost saving

8. Frequently Asked Questions (FAQs)

Q1. Is a Virtual Machine a real computer?

No.

It's a **software-based computer** running on real hardware

Q2. Is VM slower than a physical computer?

Slightly, but modern servers are very fast. Most users won't notice.

Q3. Is virtualization same as cloud?

No.

- Virtualization = technology
- Cloud = service built using virtualization

Q4. What happens if the physical server fails?

All VMs stop.
That's why backups and clusters are used

Q5. Can students use virtualization?

Yes.
Tools like VirtualBox are perfect for learning.

Q6. Can I create a VM on my laptop?

Yes.
Use:

- VirtualBox
- VMware Workstation

Q7.Do containers replace VMs completely?

No. They **work together**.
Common real-world setup:

- Physical Server
- Virtual Machine
- Docker Containers inside VM

Why?

- VM gives isolation and security
- Containers give speed and scalability

"VMs virtualize hardware, containers virtualize applications."

Real-time example (with VM)
Example: Company production server

- Cloud provider gives you a VM
- You install Docker on that VM
- You run multiple Spring Boot services as containers

docker run my-user-service
docker run my-order-service
docker run my-payment-service

Q8. When to use what?

Use Virtual Machines when:

- Need strong isolation
- Running different OS types
- Legacy applications

Use Containers when:

- Microservices

- Frequent deployments
- Fast scaling
- Cloud-native apps

Virtual Machine

Hardware
└── VM
  └── OS
    └── App

Container

Hardware
└── OS
  └── Container
    └── App

Q9. Why are containers faster than VMs?

Answer:
Containers do not boot a separate operating system. They share the host OS kernel, so they start in seconds, whereas VMs need to boot a full OS and take minutes.

Q10: Can we use containers without VM ?

Yes, containers can run directly on a physical machine without a VM.
However, in production, containers usually run inside VMs to get better isolation, security, and cloud compatibility.

Q11. What does "moving application from VM to Cloud" really mean?

People don't actually move **from VM to Cloud**.
They move from:
👉 **VMs they manage themselves**
to
👉 **VMs managed by a Cloud provider**
Cloud still uses VMs under the hood.
The difference is **who manages what**

Before (traditional VM setup)

- You own or rent a server
- You create a VM
- You install OS, Java, Docker, monitoring
- You handle:
  - Server crash
  - Disk full
  - OS patching
  - Scaling

Everything is **your responsibility**.

After (Cloud setup)

- Cloud provider gives you ready infrastructure

- You deploy your app
- Cloud handles:
    - Hardware
    - Power
    - Network
    - VM availability

You focus more on **application**, less on **machines**.

Mindset change:
"I manage servers" → "I manage services"

Interview-ready one-liner (very important)

Say this confidently:

> Cloud does not remove virtual machines. It removes the operational burden of managing them.

That's a **golden answer**.

Q12. VM vs Cloud comparison (simple)

Aspect	VM (self-managed)	Cloud
Hardware	You manage	Cloud manages
Scaling	Manual	Easy / automatic
Cost	Fixed	Pay-as-you-use
Reliability	Limited	High
Focus	Infra + app	Mostly app

One final analogy to remember forever

- **VM** = owning a vehicle (maintenance is yours)
- **Cloud** = using a cab service (just ride and pay)

Same destination. Very different effort.

Docker

22 December 2025 13:20

First, imagine this real life problem
You bake a cake at home.
It tastes perfect.
You take the same recipe to your friend's house.
The cake fails.
Why?

- Oven temperature is different
- Ingredients brand is different
- Tools are different

Same recipe, different result.
This is **exactly** what happens with software.

What is Docker? (very simple)

Docker is a tool that packs an application together with everything it needs to run.

That includes:

- The app itself
- Programming language (Java, Node, Python)
- Libraries
- Settings

All packed into one **container**.
So wherever you run it:

- Laptop
- School computer
- Office server
- Cloud

👉 It behaves the same everywhere.

Important Docker terms (explained simply)

1. Container

A **container** is a running box that has:

- Your app
- Everything it needs

2. Image

An **image** is a blueprint or recipe.

- You **build** an image
- You **run** an image to create a container

Analogy:

- Image = Cake recipe
- Container = Actual baked cake

3. Docker Engine

This is the **machine that runs containers**.

4. Dockerfile

A **Dockerfile** is a step-by-step instruction.
Example:

- Use Java
- Copy my app
- Start the app

5. Docker Hub

An online store for images.

- Java image
- MySQL image
- Redis image

Analogy:

Play Store for apps.

6. Port

Port is how your app talks to the outside world.
Example:

- App runs inside container on port 8080
- Browser uses that port to talk

Analogy:

Door number of a house.

7. Volume

Containers forget data when stopped.
Volume is used to **remember data**.

8. Docker Compose

Used when many containers work together.
Example:

- App
- Database
- Cache

All started using **one command**.
Analogy:

Remote control for TV, AC, and speaker together.

Advantages:

Easy sharing

You can share your app as a container.
Others don't need to install Java, DB, anything.

4. Faster learning and testing

You can try:

- MySQL
- Redis
- Kafka

Without installing them forever on your system.

A super simple real example

Without Docker

Your teacher says:

Install Java, MySQL, set environment variables, then run the app.

Takes hours.

With Docker

Your teacher says:

docker run my-school-app
App runs in seconds.

One sentence that locks this in

Docker **packages the machine with the app**.
VM **runs the app directly on the machine**.

What problem Docker Compose solves (first understand this)

Imagine your app needs:

- Spring Boot app
- MySQL database
- Redis cache

Without Docker Compose:

- Start MySQL container
- Start Redis container
- Start app container
- Make sure all are connected
- Remember ports, names, env variables

Messy and error-prone.

👉 **Docker Compose solves this by starting everything together using one file and one command**

One-line definition (easy to remember)

Docker Compose is a tool to run multiple Docker containers together using one configuration file.

How app talks to MySQL (very important concept)

Each service = one container.
Inside Docker Compose:

- Containers talk using **service names**
- Not localhost

So in Spring Boot config:

```
spring.datasource.url=jdbc:mysql://mysql:3306/schooldb
spring.datasource.username=root
spring.datasource.password=root
```

👉 mysql is the service name, not IP.
This is **built-in Docker networking**.

Example:

```
version: "3.8"

services:
  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: schooldb
    ports:
      - "3306:3306"

  app:
    image: my-school-app
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: dev
    depends_on:
      - mysql
```

From the folder containing docker-compose.yml:

This is what ports: 8080:8080 actually means

```
ports:
  - "8080:8080"
```

Read it like this:

Laptop port 8080 → Container port 8080

Or in words:

"If someone comes to my laptop on port 8080, forward them to port 8080 inside this container."
Now the browser can reach the app.

Running everything (the magic command):

docker compose up
Docker will:

1. Pull MySQL image
2. Start MySQL container
3. Start app container
4. Connect them automatically

To stop everything:

docker compose down

The biggest mental shift (important)

In VM world

"Everything runs on the same machine, so use localhost."

```
Physical Server
  |
  └── Virtual Machine (1 OS, 1 IP)
      ├─ Angular     → localhost:4200
      ├─ Spring Boot → localhost:8080
      └─ MySQL       → localhost:3306
```

Why localhost works in VM

Inside a VM:

- localhost means → **this VM**
- All apps are running on the same OS
- Ports are just different doors on the same machine

In Docker world

"Everything runs in its own box, so use service names."
Once this clicks, Docker networking becomes easy

```
One Server / One VM
  |
  └── Docker
      ├─ Angular container   (its own localhost)
      ├─ Backend container   (its own localhost)
      └─ MySQL container     (its own localhost)
```

Real example (step by step)

Case 1: Only EXPOSE

EXPOSE 8080

docker run my-app
Result:
❌ Browser cannot access
❌ App is running, but hidden

Case 2: Port mapping

docker run -p 8080:8080 my-app
Result:

☑ Browser → laptop:8080
☑ Laptop → container:8080
☑ App reachable

Why not always expose ports then?

Because:

- Databases should not be public
- Internal services should stay internal
- Security matters

That's why:

- Frontend → ports exposed
- Backend → sometimes exposed
- Database → usually NOT exposed

Docker Daemon:

When we install docker, we're actually installing docker daemon. The heart of docker is docker daemon. When we execute docker command , the docker daemon is managing and listening to the containers.

Commands:

- Install Docker Desktop (Windows)
- docker --version
- docker info
- docker help

Important commands:

- docker pull
- docker images
- docker ps
- docker ps -a
- docker stop
- docker rm
- docker rmi

Kubernetes

23 December 2025 19:21

If you've ever used an app like Netflix, Spotify, or Instagram, you've used a system managed by **Kubernetes**.

What is Kubernetes? (The "Super-Manager")

Kubernetes (often called **K8s**) is an open-source system that acts like that Super-Manager. Its name actually comes from the Greek word for **"Helmsman"** or **"Captain."**

If **Docker** creates the boxes, **Kubernetes** is the captain of the ship that decides:

1. **Where** to put the boxes (Scheduling).
2. **How many** boxes we need (Scaling).
3. What to do if a box **breaks** (Self-healing).

Why is it "Needed" and "Important"?

Without Kubernetes, big companies would have to hire thousands of people just to "watch" servers. Kubernetes does this automatically through four main powers:

- **Self-Healing:** If a container crashes at 3:00 AM, Kubernetes notice it's gone and instantly starts a new one. The users never even notice.
- **Horizontal Scaling:** If a million people suddenly visit your site (the "Taylor Swift Ticket Effect"), Kubernetes sees the "heat" and automatically spins up 100 more containers to handle the crowd.
- **Load Balancing:** It acts like a traffic cop, sending visitors to the containers that aren't busy so no single "kitchen" gets overwhelmed.
- **Zero Downtime Updates:** When you want to update your app, Kubernetes replaces the old "boxes" with new ones one-by-one. If the new version has a bug, it automatically switches back to the old one.

2. Real-Time Scenario: The "Black Friday" Shopping Surge

To see hundreds of containers working in real-time, look at a giant e-commerce site like **Amazon** or **Target** during a major sale.

The Setup (Normal Day)

On a Tuesday morning, the website might be running on **100 containers**:

- **50 Containers** for "Searching/Browsing" (looking at products).
- **30 Containers** for "Inventory" (checking if items are in stock).
- **20 Containers** for "Checkout" (processing credit cards).

The Surge (The Sale Starts)

At Midnight, a huge "Limited Edition" sneaker drops. Suddenly, 1 million people hit the site.

1. **The Trigger:** Kubernetes monitors the "CPU temperature" (the stress) of the containers. It sees the 50 "Searching" containers are at 95% capacity.
2. **The Scaling:** In seconds, Kubernetes screams, *"We need more power!"* and spins up **400 more containers**.
3. **The Balance:** Now you have 500 containers running. Kubernetes acts like a traffic cop, diverting the 1 million users evenly across all 500 so the website doesn't crash.
4. **The Cleanup:** When the sale ends and people go to sleep, Kubernetes realizes, *"We don't need 500 containers anymore; that's expensive."* It shuts down 400 of them to save money.

That is a great pivot. Let's move from a single pizza shop to a **Massive Global Restaurant Brand** (like a McDonald's or Chipotle) to show how this works at scale.

1. The Better Analogy: The Automated Mega-Restaurant

In this world, we don't have human cooks; we have **Robot Kitchen Pods**.

The Docker Container (The "Kitchen Pod")

Imagine a high-tech shipping container that is a fully functional, miniature kitchen. Inside, it has exactly what it needs: the stove, the ingredients, and the software to cook one specific thing (e.g., "The Burger Bot").

- **The Benefit:** It doesn't matter if you put this pod in the middle of New York or on a beach in Hawaii; it will produce the exact same burger every time because everything it needs is **contained** inside.

Kubernetes (The "Restaurant AI Manager")

Now, imagine you have 500 of these "Kitchen Pods" spread across a city. You can't manually check on them. You install **Kubernetes** as the "AI Manager" sitting in a control tower.

- **The "Desired State":** You tell the AI, *"I want 10 Burger Pods running at all times."*
- **Self-Healing:** If a lightning strike hits Pod #7 and it dies, Kubernetes doesn't panic. It immediately sees "I have 9 pods, but the boss wants 10," and it automatically drops a brand-new Pod #7 into place.
- **The Service (The Waiter):** Customers don't talk to the pods directly. They talk to a **"Service"** (a waiter). The waiter takes the order and gives it to whichever Pod is least busy.

2. Real-Time Scenario: The "Black Friday" Shopping Surge

To see hundreds of containers working in real-time, look at a giant e-commerce site like **Amazon** or **Target** during a major sale.

The Setup (Normal Day)

On a Tuesday morning, the website might be running on **100 containers**:

- **50 Containers** for "Searching/Browsing" (looking at products).
- **30 Containers** for "Inventory" (checking if items are in stock).
- **20 Containers** for "Checkout" (processing credit cards).

The Surge (The Sale Starts)

At Midnight, a huge "Limited Edition" sneaker drops. Suddenly, 1 million people hit the site.

1. **The Trigger:** Kubernetes monitors the "CPU temperature" (the stress) of the containers. It sees the 50 "Searching" containers are at 95% capacity.
2. **The Scaling:** In seconds, Kubernetes screams, *"We need more power!"* and spins up **400 more containers**.
3. **The Balance:** Now you have 500 containers running. Kubernetes acts like a traffic cop, diverting the 1 million users evenly across all 500 so the website doesn't crash.

4. **The Cleanup:** When the sale ends and people go to sleep, Kubernetes realizes, *"We don't need 500 containers anymore; that's expensive."* It shuts down 400 of them to save money.

3. Summary: Why we "Need" it

Without Kubernetes, a human engineer would have to:

1. Manually log in to a server.
2. Start a new container.
3. Update the traffic settings.
4. Do this 400 times while the website is crashing.

Kubernetes turns a "Manual Emergency" into a "Silent Background Task."

Restaurant Term	Kubernetes Term	What it actually is
The Kitchen Pod	**Pod**	The smallest unit (usually holds 1 Docker container).
The Plot of Land	**Node**	A physical server or computer where Pods live.
The Restaurant AI	**Control Plane**	The "brain" that makes decisions.
The Waiter	**Ingress / Service**	The way users find and talk to your containers.

Why we need 30 containers for search operation?

Now suddenly:

- **10,000 users** search at the same time
- Everyone types different things:
  - phone
  - charger
  - laptop
  - shoes
  - headphones

Important truth (very important 👇)

👉 **One container can handle only limited requests at a time**

Let's say:

- **1 container can handle 50 searches per second**

Now calculate:

- Users hitting at same second = **10,000**
- Capacity = **50**

❌ 9,950 users are waiting
❌ App becomes slow
❌ People close the app
❌ Business loses money

One Spring Boot API → many Pods

Now imagine:

- 50 Pods running
- All expose /search

To you, it still looks like:

GET /search?q=headphones
You never care **which pod handles it**.
Kubernetes handles that.

Scaling explained using Spring Boot logic

Let's say:

- One Spring Boot app handles ~100 requests/sec
- Traffic = 5,000 requests/sec

Math:

- Needed instances = 50

Kubernetes does this automatically:

- Starts 50 Spring Boot containers
- Stops extras when traffic reduces

You did **not change your controller code**.

Health checks using Spring Boot Actuator

Spring Boot already gives:

/actuator/health
Kubernetes keeps checking it.
If response is:

{ "status": "DOWN" }
Kubernetes:

- Stops sending traffic
- Replaces the Pod

Spring Boot + Kubernetes = perfect match.

Step 4: The supermarket counter analogy 🛒

Think like this:

- One cashier can handle **one customer at a time**
- If 100 people come:
  - One cashier = chaos
  - 10 cashiers = smooth flow

Same logic.

In software:

- **Cashier = container**
- **Customer = request**

Why Kubernetes instead of humans?

Without Kubernetes:

- Engineer must manually add servers
- By the time they act → sale is over

With Kubernetes:

- Automatic
- Fast
- Smart
- Reversible

After sale:

- Containers reduce back to 2 or 3
- Money saved 💰

One Kubernetes Pod can contain multiple containers.
But it is done only in specific cases, not by default.

- Container 1: Spring Boot application
- Container 2: Log shipper (reads logs and sends to Elasticsearch)
  Both must:
- Start together
- Stop together
- Live on the same machine
  So they go into **one Pod**.

Why Kubernetes didn't just use containers directly

Problem 1: Containers don't naturally work in groups

Real applications often need:

- One main app
- One helper (logging, proxy, config watcher)

If Kubernetes managed containers individually:

- They could start at different times
- They could land on different machines
- They wouldn't reliably talk over localhost

Kubernetes needed a **guarantee**:
"These containers must live and die together."
That guarantee is a **Pod**.

Problem 2: Shared storage is hard without Pods

Many helpers need access to:

- App logs
- Config files
- Temporary data

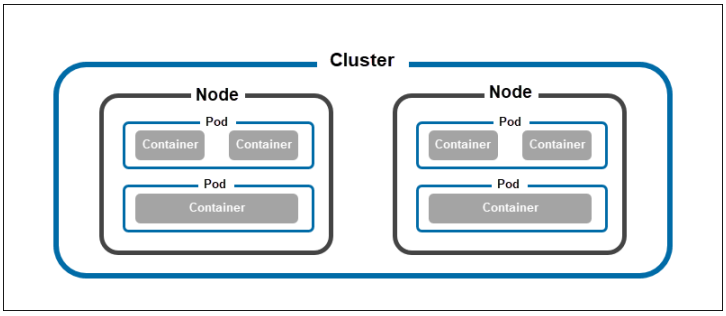
Pods provide **shared volumes** easily.
Containers alone cannot coordinate this cleanly.

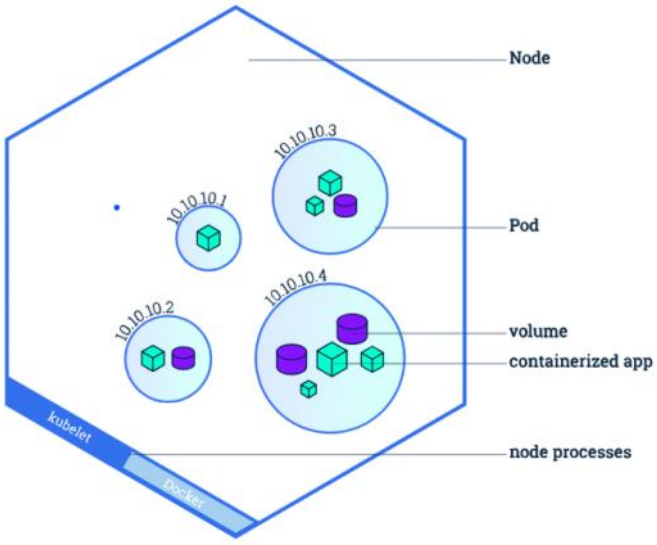
Problem 3: Scheduling and scaling need a smallest unit

Kubernetes schedules workloads on nodes.
It needed **one atomic unit**:

- Schedule together
- Restart together
- Scale together

That unit is the Pod, not the container.
In short
Kubernetes uses containers to run code,
but it uses **Pods to manage reality**.





Events (MOST USEFUL FOR DEBUGGING)

This is the **timeline** of what happened.

When we use "**kubectl describe pod hello-world**"

Events

Type	Reason	Age	From	Message
Normal	Scheduled	3m38s	default-scheduler	Successfully assigned sn-labs-mohamedashf2/hello-world to 10.241.64.23
Normal	Pulling	3m38s	kubelet	Pulling image
Normal	Pulled	3m35s	kubelet	Successfully pulled image in 2.453s
Normal	Created	3m35s	kubelet	Created container: hello-world
Normal	Started	3m35s	kubelet	Started container hello-world
Warning	FailedToRetrieveImagePullSecret	5s (x6)	kubelet	Unable to retrieve some image pull secrets (dh)

Scheduled → Pulling → Pulled → Created → Started

Read it like a story:

1. **Scheduled**
   - Kubernetes chose a node
2. **Pulling**
   - Image downloaded
3. **Pulled**
   - Image download successful
4. **Created**
   - Container created
5. **Started**
   - App started successfully

Perfect flow ☑

Pod to Pod communication happens through CNI (Container Network Interface)

Interview-ready summary

Why not containers directly?

Kubernetes needed a higher-level unit that could group containers, share networking and storage, and be scheduled as one atomic unit. That unit is the Pod.

Why single-container Pods are preferred?

They follow the single-responsibility principle, scale independently, simplify operations, and cover most real-world use cases.

kubectl describe pod hello-world

```
Name:             hello-world
Namespace:        sn-labs-mohamedashf2
Priority:         1
Priority Class Name:  normal
Service Account:  default
Node:             10.241.64.23/10.241.64.23
Start Time:       Wed, 24 Dec 2025 00:29:05 -0500
Labels:           run=hello-world
Annotations:      cni.projectcalico.org/containerID: 153fe416b108023082cf64904c3c26ecd6dd34071254b0764e8226844218350b
                  cni.projectcalico.org/podIP: 172.17.173.140/32
                  cni.projectcalico.org/podIPs: 172.17.173.140/32
                  kubernetes.io/limit-ranger:
                    LimitRanger plugin set: cpu, ephemeral-storage, memory request for container hello-world; cpu, ephemeral-storage, memory limit
for contain...
Status:           Running
IP:               172.17.173.140
IPs:
  IP:  172.17.173.140
Containers:
  hello-world:
    Container ID:   containerd://5d57e751bd197cd2b7765418c248126b2a7dde5a648d0e71808d74f8218a2f2c
    Image:          us.icr.io/sn-labs-mohamedashf2/hello-world:1
    Image ID:       us.icr.io/sn-labs-mohamedashf2/hello-
world@sha256:4030a06ef5cbc8cd7bda9270f71124a0dd86ab6257b75144cf71a5b8db36ee03
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Wed, 24 Dec 2025 00:29:08 -0500
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:              500m
      ephemeral-storage:  5Gi
      memory:           512Mi
    Requests:
      cpu:              200m
      ephemeral-storage:  512Mi
      memory:           128Mi
    Environment:        <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-2x5kz (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-2x5kz:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    ConfigMapOptional:       <nil>
    DownwardAPI:             true
QoS Class:                   Burstable
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 600s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 600s
Events:
  Type    Reason                  Age              From               Message
  ----    ------                  ----             ----               -------
  Normal  Scheduled               3m38s            default-scheduler  Successfully assigned sn-labs-mohamedashf2/hello-world to 10.241.64.23
  Normal  Pulling                 3m38s            kubelet            Pulling image "us.icr.io/sn-labs-mohamedashf2/hello-world:1"
```

```
  Normal   Pulled                  3m35s            kubelet          Successfully pulled image "us.icr.io/sn-labs-mohamedashf2/hello-world:1" in 2.453s
(2.453s including waiting). Image size: 27780427 bytes.
  Normal   Created                 3m35s            kubelet          Created container: hello-world
  Normal   Started                 3m35s            kubelet          Started container hello-world
  Warning  FailedToRetrieveImagePullSecret  5s (x6 over 3m38s)  kubelet          Unable to retrieve some image pull secrets (dh); attempting to pull
the image may not succeed.
theia@theiadocker-mohamedashf2:/home/project/CC20
```

Got it 👍
Below is **only your pasted content**, reformatted into **clean tables**.
No extra explanation. No added data.

Pod Overview

Field	Value
Name	hello-world
Namespace	sn-labs-mohamedashf2
Priority	1
Priority Class Name	normal
Service Account	default
Node	10.241.64.23 / 10.241.64.23
Start Time	Wed, 24 Dec 2025 00:29:05 -0500
Status	Running
QoS Class	Burstable

Labels

Key	Value
run	hello-world

Annotations

Key	Value
cni.projectcalico.org/containerID	153fe416b108023082cf64904c3c26ecd6dd34071254b0764e8226844218350b
cni.projectcalico.org/podIP	172.17.173.140/32
cni.projectcalico.org/podIPs	172.17.173.140/32
kubernetes.io/limit-ranger	LimitRanger plugin set: cpu, ephemeral-storage, memory request and limit

Networking

Field	Value
IP	172.17.173.140
IPs	172.17.173.140
Port	<none>
Host Port	<none>

Container Details

Field	Value
Container Name	hello-world
Container ID	containerd://5d57e751bd197cd2b7765418c248126b2a7dde5a648d0e71808d74f8218a2f2c
Image	us.icr.io/sn-labs-mohamedashf2/hello-world:1
Image ID	us.icr.io/sn-labs-mohamedashf2/hello-world@sha256:4030a06ef5cbc8cd7bda9270f71124a0dd86ab6257b75144cf71a5b8db36ee03
State	Running
Started	Wed, 24 Dec 2025 00:29:08 -0500

Ready	True
Restart Count	0
Environment	<none>

Resource Limits

Resource	Limit
CPU	500m
Ephemeral Storage	5Gi
Memory	512Mi

Resource Requests

Resource	Request
CPU	200m
Ephemeral Storage	512Mi
Memory	128Mi

Mounts

Mount Path	Volume	Access
/var/run/secrets/kubernetes.io/serviceaccount	kube-api-access-2x5kz	Read-only

Pod Conditions

Type	Status
PodReadyToStartContainers	True
Initialized	True
Ready	True
ContainersReady	True
PodScheduled	True

Volumes

Name	Type	Details
kube-api-access-2x5kz	Projected	TokenExpirationSeconds: 3607, ConfigMap: kube-root-ca.crt, DownwardAPI: true

Node Scheduling

Field	Value
Node Selectors	<none>
Tolerations	node.kubernetes.io/not-ready:NoExecute (600s)node.kubernetes.io/unreachable:NoExecute (600s)

Events

Type	Reason	Age	From	Message
Normal	Scheduled	3m38s	default-scheduler	Successfully assigned sn-labs-mohamedashf2/hello-world to 10.241.64.23
Normal	Pulling	3m38s	kubelet	Pulling image
Normal	Pulled	3m35s	kubelet	Successfully pulled image in 2.453s
Normal	Created	3m35s	kubelet	Created container: hello-world
Normal	Started	3m35s	kubelet	Started container hello-world
Warning	FailedToRetrieveImagePullSecret	5s (x6)	kubelet	Unable to retrieve some image pull secrets (dh)

OpenShift

25 December 2025 12:25

OpenShift (Enterprise Grade Kubernetes)
OpenShift is **Kubernetes + safety + tools + ease of use.**
Think of it like this:

- Kubernetes is a **raw engine**
- OpenShift is a **ready-to-drive car**

You still use Kubernetes underneath, but OpenShift makes life easier.

Companies take the open source kubernates and develop their own distribution or own flavour of kubernates.
Like
RedHat (OpenShift)
EKS (Amazon)
AKS (Azure) etc

Managed Services:
Red Hat collaborates with major cloud providers to offer managed OpenShift services, offloading the management overhead from the user. These include:

- ROSA (Red Hat OpenShift Service on AWS)
- ARO (Azure Red Hat OpenShift)
- Managed services on Google Cloud Platform and IBM Platform

Simple real-life analogy

Kubernetes

- You are given a kitchen
- Raw vegetables
- Stove parts
- You must cook everything yourself

OpenShift

- Kitchen is ready
- Stove is installed
- Ingredients are cleaned
- Recipes are provided

You just focus on cooking.

What OpenShift adds on top of Kubernetes
OpenShift includes Kubernetes, **plus**:

1. **Easy dashboard**
   - Click buttons instead of typing commands
2. **Built-in security**
   - Apps cannot run with dangerous permissions
   - Safer by default
3. **Built-in CI/CD**
   - Automatically builds and deploys code
4. **Developer-friendly**
   - Developers push code
   - OpenShift builds and runs it
5. **Enterprise support**
   - Companies get official support from Red Hat

Difference between Kubernetes and OpenShift (easy table)

Topic	Kubernetes	OpenShift
What it is	Core platform	Kubernetes with extras
Difficulty	Hard for beginners	Easier to use

Security	You configure it	Secure by default
UI	Optional	Built-in web console
Company support	Community-based	Enterprise support
Best for	Platform experts	Companies & teams

Why companies use OpenShift

Companies choose OpenShift because:

- Less setup headache
- Better security
- Faster development
- Works well in banks and large enterprises

That’s why many big companies prefer it.

Why avoid raw Kubernetes?

Lack of direct support: If issues or questions arise, reaching out to Kubernetes engineers is difficult and time-consuming because it's an open-source platform. Organizations would typically create an issue on GitHub and wait for a contributor to respond, which enterprises might not prefer

Management overhead: Kubernetes is not "opinionated," meaning it doesn't specify how or where to install it (e.g., on specific virtual machines or operating systems). This leads to significant management overhead, especially when deploying at scale with many nodes or multiple clusters across different regions and availability zones

From <https://www.youtube.com/watch?v=vwDezts_aFg&t=1101s>

Example for Docker, Kubernetes, OpenShift

25 December 2025 12:38

Let's continue the same story and use **one clear restaurant example** from start to end.

Step 1: The restaurant setup (big picture)

Imagine you want to run a **big restaurant** that serves many customers every day.
You need:

- Food recipes
- Chefs
- Kitchen management
- Safety rules
- Someone to handle rush hours

This is exactly how modern apps work.

Step 2: Where Docker fits

Docker
Docker is like **pre-packed recipe boxes**.

In restaurant terms:

- A recipe box contains:
  - Recipe
  - Ingredients
  - Cooking instructions
- Anyone can open the box and cook the same dish anywhere

In software terms:

- Docker packages:
  - App code
  - Libraries
  - Settings
- The app runs the same on any computer

👉 **Docker = packs the app neatly**

Step 3: What problem Docker solves

Without Docker:

- Dish tastes different in every branch
- Missing ingredients
- "It works in my kitchen, not yours" problem

With Docker:

- Same taste everywhere
- No surprises
- Easy to move between computers

Step 4: Why Docker alone is not enough

Now imagine:

- You have **100 recipe boxes**
- 50 chefs
- Lunch rush starts

Questions:

- Who opens which box?
- What if a chef quits?
- What if suddenly 500 customers arrive?

Docker doesn't manage this.

Docker only **packs food**. It doesn't run the restaurant.

In other words,
Now imagine:

- 1 Angular container
- 1 Spring Boot container
- Suddenly 10,000 users arrive

Questions:

- Who starts more containers?
- Who restarts if Spring Boot crashes?
- Who balances traffic?

Docker doesn't answer this.

Step 5: Enter Kubernetes

Kubernetes
Kubernetes is the **restaurant manager**.
It:

- Assigns recipe boxes to chefs
- Adds more chefs during rush
- Replaces chefs who leave
- Closes extra counters when rush is over

👉 **Kubernetes = manages many Docker containers**

Step 6: Then what does OpenShift do?

OpenShift
OpenShift is the **restaurant chain franchise system**.
It provides:

- Trained chefs
- Clean kitchen rules
- Security checks
- Management dashboard
- Customer safety rules

You don't build the restaurant from scratch.
👉 **OpenShift = Kubernetes + safety + tools**

OpenShift is Kubernetes, but **enterprise-ready**.
For your Spring Boot + Angular app, it adds:

- Built-in security rules
- Easy web dashboard
- CI/CD pipelines
- Safer defaults (very important for banks)

Step 7: All three together (simple mapping)

Restaurant World	Software World
Recipe box	Docker
Chef manager	Kubernetes
Franchise system	OpenShift
Dish	Application
Customers	Users

Step 8: One simple flow (important)

1. Developer writes code
2. **Docker** packs it into a container
3. **Kubernetes** runs and manages containers
4. **OpenShift** makes everything safer and easier for companies

Step 9: One-line takeaway

- **Docker** makes the food ready
- **Kubernetes** runs the restaurant
- **OpenShift** runs the restaurant chain properly

If you want next, I can:

- Map this to **Spring Boot + Angular**
- Explain with a **school classroom**
- Or give **interview-ready answers**

Just tell me.

In general where we'll deploy our application , in kubernetes or Openshift? i'm confused with that?

This confusion is very common. Let me clear it **once and for all**, in simple words.

Short answer (remember this)

👉 **You deploy your application on Kubernetes.**

👉 **OpenShift is a platform that already contains Kubernetes.**

So when you deploy to OpenShift, **you are still deploying to Kubernetes**, just through OpenShift.

Step 1: Think in layers (very important)

Layer 1: Where apps actually run

Kubernetes

- Kubernetes is the engine that:
  - Runs containers
  - Creates pods
  - Scales apps
  - Restarts crashed apps

👉 **Apps run on Kubernetes. Always.**

Layer 2: Who manages Kubernetes for you

OpenShift

- OpenShift is:
  - Kubernetes + security + UI + CI/CD
- It does not replace Kubernetes
- It **uses Kubernetes internally**

👉 OpenShift is a **managed, enterprise-ready Kubernetes**.

Step 2: Simple real-life analogy

Kubernetes

- Engine of a car

OpenShift

- Full car (engine + brakes + safety + dashboard)

You don’t choose *engine vs car*.

You choose **how much help you want** around the engine.

Step 3: So where do people actually deploy?

Option 1: Direct Kubernetes

Teams deploy **directly to Kubernetes** when:

- Startup or small team
- Strong DevOps skills
- Want full control
- Less strict security rules

Example:

- Self-managed Kubernetes
- EKS, AKS, GKE

Option 2: OpenShift

Teams deploy **to OpenShift** when:

- Large enterprise
- Banking, insurance, telecom
- Security is critical
- Dev teams should not manage infra

Example:

- Red Hat OpenShift
- OpenShift on AWS / Azure / On-prem

Step 4: What YOU should say in interviews (gold answer)

> Applications always run on Kubernetes.
> OpenShift is an enterprise platform built on top of Kubernetes that simplifies deployment, security, and operations. When we deploy to OpenShift, we are still deploying to Kubernetes.

This answer shows **clear understanding**.

Step 5: Very clear comparison

Question	Kubernetes	OpenShift
Where app runs	☑ Yes	☑ Yes (via Kubernetes)
Is Kubernetes inside	❌ No	☑ Yes
Security by default	❌ Manual	☑ Built-in
Enterprise support	❌ Optional	☑ Yes
Used by banks	⚠ Rare	☑ Very common

Step 6: Remove the confusion completely

❌ "Should I deploy to Kubernetes or OpenShift?"
☑ **Wrong question**
☑ "Do we use plain Kubernetes or OpenShift-managed Kubernetes?"
☑ **Correct question**

Final one-line takeaway (memorize this)

> Kubernetes is where applications run. OpenShift is how enterprises manage Kubernetes safely and easily.

FAQ

25 December 2025 12:47

1) Where we deploy our application in kubernetes or openshift?

👉 **You deploy your application on Kubernetes.**
👉 **OpenShift is a platform that already contains Kubernetes.**
So when you deploy to OpenShift, **you are still deploying to Kubernetes**, just through OpenShift.

2) Why do we need Docker when we already have a server?

Answer:
Servers differ in OS, libraries, and configs. Docker removes this mismatch by packaging everything together, which avoids the “works on my machine” problem.

3) What problem does Kubernetes solve on top of Docker?

Answer:
Docker runs containers, but Kubernetes manages them at scale. It handles auto-scaling, self-healing, load balancing, and rolling updates.

How does Kubernetes handle traffic for Spring Boot services?

Answer:
Kubernetes exposes Spring Boot containers using Services and routes traffic evenly across multiple pods using built-in load balancing.

5) What is a Pod in Kubernetes?

Answer:
A Pod is the smallest deployable unit in Kubernetes. It usually contains one container, like a Spring Boot app, and shares networking and storage.

What is OpenShift and how is it related to Kubernetes?

Answer:
OpenShift is an enterprise platform built on top of **Kubernetes**. It adds security, developer tools, CI/CD, and an easy UI.

Can Spring Boot run without Docker in Kubernetes?

Answer:
No. Kubernetes runs containers, not raw applications. Spring Boot must be packaged as a Docker image before Kubernetes can run it.

How does scaling work for Spring Boot in Kubernetes?

Answer:
Kubernetes increases or decreases the number of Spring Boot pods based on CPU, memory, or traffic using Horizontal Pod Autoscaler.

How does OpenShift improve developer experience?

Answer:
It provides:

- Web UI
- Git-based deployments
- Built-in CI/CD
- Easy log and pod monitoring

Typical real-world architecture answer

Answer:
Angular frontend and Spring Boot backend are containerized using **Docker**, orchestrated by Kubernetes for scaling and reliability, and deployed on OpenShift for enterprise security and ease of operations.

How does traffic reach a Spring Boot app in Kubernetes?

Answer:
Through a Service or Ingress, which routes external traffic to the correct pods.

. How do you debug issues in Kubernetes?

Answer:
By checking pod logs, events, health probes, and resource usage.

Scenario 1: App works locally but fails in production

Question:
Your Spring Boot app runs fine locally but fails after deployment to Kubernetes. What do you check?
Answer:
First I check pod logs and events. Then I verify environment variables, ConfigMaps, Secrets, container port vs service port, and resource limits. Most failures are due to missing configs or wrong ports.

Scenario 2: Spring Boot pod keeps restarting

Question:
A Spring Boot pod is restarting again and again. How do you debug?
Answer:
I check pod logs to see startup errors, verify health probes, and check memory limits. If memory is too low, the pod may be getting killed.

Scenario 3: High traffic suddenly hits the app

Question:
Your app suddenly gets 10x traffic. What happens in Kubernetes?
Answer:
Kubernetes scales the Spring Boot pods using Horizontal Pod Autoscaler. Traffic is load-balanced automatically across new pods.

Scenario 4: Zero-downtime deployment requirement

Question:
How do you deploy a new version without downtime?
Answer:
Using rolling updates. Kubernetes brings up new pods gradually and removes old ones only after the

new pods are ready

Scenario 5: Config differs between dev and prod

Question:
How do you manage different configs for dev, QA, and prod?
Answer:
Using ConfigMaps for normal config and Secrets for sensitive values. Same Docker image is used across environments.

Scenario 6: One pod behaves differently from others

Question:
One Spring Boot pod behaves differently. Why?
Answer:
Possible reasons are corrupted config, uneven traffic, node issues, or stale cache. I compare logs and environment variables across pods.

Scenario 7: Docker image size is very large

Question:
Your Docker image is too large. How do you fix it?
Answer:
Use multi-stage builds, smaller base images, and avoid unnecessary files. Smaller images start faster and deploy quicker.

Scenario 8: Rolling back a bad deployment

Question:
New deployment is broken. What do you do?
Answer:
Rollback to the previous stable version using deployment history. Kubernetes supports quick rollbacks.

Scenario 9: CI/CD pipeline question

Question:
How does code go from Git to production?
Answer:
Code is built, Docker image is created, image is pushed to registry, Kubernetes deploys it, and OpenShift automates and monitors the process.

Quick Tip for Interviews

If stuck, always mention:

- Logs
- Health checks
- Configs
- Resource limits
- Scaling

Interviewers want to see **system thinking**, not just commands.

NgInx

25 December 2025 13:59

What is Nginx?

Nginx (pronounced “engine-x”) is a **traffic controller for websites**.
In simple words:

- It sits in front of your application
- It receives requests from users (browser)
- It decides what to do with those requests

Nginx is a high-performance web server and reverse proxy used to serve static content, route requests, and improve performance and security.

Why use Nginx with Angular?
Angular produces static files, and Nginx is optimized to serve static content efficiently and securely

What exactly does Nginx do?

1 Serves static files (Angular is static)

Angular build output:

index.html
main.js
styles.css
assets/
Nginx is **excellent** at serving these files very fast.

2 Acts as a Reverse Proxy (most important)

User thinks:

example.com
Reality:

example.com → Nginx → Angular / Spring Boot

User never sees:

- Port numbers
- Internal servers
- Backend details

3 Load Balancing (when users increase)

Browser
↓
Nginx
↓ ↓ ↓
Backend-1 Backend-2 Backend-3

Nginx distributes traffic evenly.

4 Security & SSL (HTTPS)

- Handles HTTPS
- Blocks suspicious requests
- Protects backend

Angular project example (very practical)

Scenario

You have:

- Angular app (Frontend)
- Spring Boot API (Backend)

Without Nginx:

Angular → http://localhost:4200
API → http://localhost:8080
Problems:

- CORS issues
- Exposed ports
- Hard to deploy

With Nginx (best practice)

User → https://myapp.com
↓
Nginx
↓ ↓
Angular Spring Boot

User sees **only one URL**.

Where is Nginx commonly used?

- Hosting Angular / React apps
- Microservices architecture
- Docker + Kubernetes
- Cloud deployments
- High-traffic websites

Almost every production web app uses Nginx or something similar.

Web servers:
Apache Httpd
NgInx (It can handle large number of users and consume slow memory)

NgInx is a powerful web server and uses non-threaded, event-driven architecture.

It can also do othe rimportant things such as.

1. Load balancing.
2. HTTP Caching.
3. Reverse Proxy.

Proxy Server (VPN):

In order to access some website whixh maybe restricted direct access, we can use VPN to connect to the Website on our behalf.

Forward Proxy:
Multiple users -> VPN -> Server.

Reverse Proxy:
If we have one client and multiple servers, then it's reverse proxy.
Multiple servers -> VPN -> User.

User does't know whixh server I'm supposed to forward my request, hence it sends to the main VPN Server. From then it chooses.

That's where NgInx comes into picture.

Single User -> NgInx (Web server) -> Multiple Server.

Advantages:

1. **Can handle 10000 concurrent request**
2. **Cache http request**
3. **Act as Reverse Proxy, Load Balancer, API Gateway**
4. **Serve and cache static file like images, videos etc.**
5. **Handle SSL certificate.**
6. **It also handles rate Limiting, firewall, security features and TLS.**

Simple real-life analogy 🏢

Apartment complex

- **Docker** = packing each family into a flat
- **Kubernetes** = apartment manager
    - Allocates flats
    - Replaces broken lifts
    - Adds more flats if crowd increases
- **Nginx** = main gate security
    - Decides where visitors go
    - Controls entry

No visitor goes directly to a flat.

How do Docker, Kubernetes, and Nginx work together?
Docker packages applications, Kubernetes orchestrates and scales those containers, and Nginx acts as the entry point that routes and load-balances traffic.

Why Nginx when Kubernetes already exists?
Kubernetes manages containers, but Nginx handles HTTP routing, SSL, and efficient request handling.

Is Nginx mandatory in Kubernetes?
Not mandatory, but almost always used through an Ingress Controller.

One diagram to remember in your head

User
↓
Nginx (Ingress)
↓
Kubernetes Services
↓
Docker Containers (Pods)

Scalar Docker nd Kubernetes Notes

29 December 2025 19:59

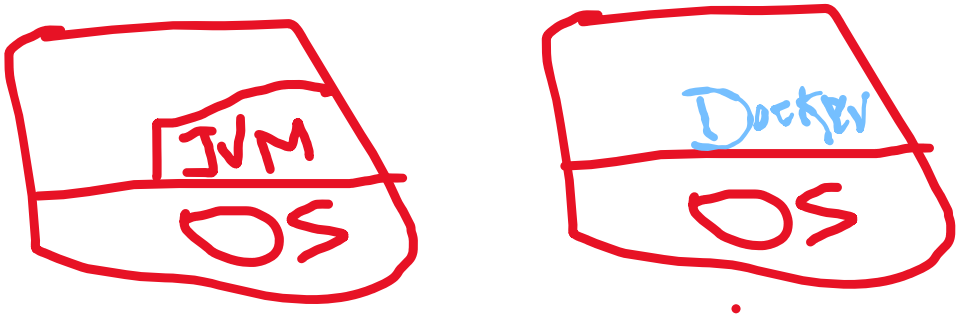
SDLC
Dockerless world problem
Docker
Kbernetes.

Development and deployment phase.

Application Dependency. (package.json)
External Dependency. (java, env variables etc)

WORA (Write once run anywhere)

James Gosling faced this problem hence java used class files (Binary or bytecode).



Should have to follow proper process.

We provide all that instructions in a file called docker file.

Docker -> will use -> DockerFile -> build -> Docker Images

Running docker image instance is called as Container.

We can store those docker images in the docker registry or docker repository like dockerHub. Just like github repository.

Companies use their own registries in their cloud. Like

AWS -> ECR (Elastic container registry)
Jfrog -> Artifactory

Image Tag (Unique Identifier) by default it uses latest.

App1:latest
App1:v1.0.0

Version will be immutable. (Always Do not rely on latest)
We can't override the the image with sam eversion again.

We can use docker labs to show demo.

Docker compose.

Consider if we have to run the db forst and tehn run the backend and tehn frontend, and we need to proide propr port an other config. We can do that using docker compose easily.

Docker -> Microservices -> (product page, review page, checkout page, payment page)

We'll create docker images for each page.

Consider we need 5 conatiners for product, 3 for cart page etc. in two different servers.
If server 1 completely stops due to some error, the server 2 will take the request.

But there will lot of server load no, this is where kubernetes comes into picture. (it will ensure the process runs smoothly like if one server crashed then it will create a new one)

Hence we need an orchestrator to perform this.

(Now how do we check the logs and monitoring for this)?